DSAI Project Report

Automatic extraction of semantic AOIs from heterogeneous SVG visualizations

Charbel ABOU KHEIR
charbel.aboukheir@telecom-paris.fr

Arsène MALLET
arsene.mallet@telecom-paris.fr

Jean Paul AL AM
jean.alam@telecom-paris.fr

Marie-José TANNOURY
marie-jose.tannoury@telecom-paris.fr

Alain TRAD
alain.trad@telecom-paris.fr

Under the supervision of

Anne-Flore Cabouat
anne-flore.cabouat@inria.fr

Abstract

This report details the development of an automated data pipeline designed to extract positional and semantic information from Areas of Interest (AOIs) within Scalable Vector Graphics (SVG) visualizations. Serving as a foundational step for a broader eye-tracking research initiative, the project aims to map raw graphical elements to their underlying semantic meaning to better interpret user cognitive processing. We implemented a modular, Python-based architecture consisting of Document Object Model (DOM) parsing, precise geometric bounding using the Shapely library, and semantic labeling. A major technical challenge involved generalizing this semantic extraction across heterogeneous SVG sources. After iterative testing of rigid hard-coded rules and volatile confidence scoring mechanisms, we developed a robust, deterministic five-step priority chain that relies on explicit attributes and structural context. Validated on 20 distinct visualizations, including both hand-crafted data and external Observable Plot charts, the final pipeline successfully maps complex vector geometries to generalized semantic categories. The resulting structured JSON outputs and custom browser-based visualizer provide decent spatial and semantic foundations required for future correlation with human gaze patterns.

Keywords: Area Of Interest (AOI), Scalable Vector Graphics (SVG), Extraction, Semantic

June 2026

I Introduction, Hypotheses & Motivations

Given a predefined set of **S**calable **V**ector **G**raphics visualizations (maps, treemaps, scatterplots, ETC..), this project aims to build a set of tools capable of extracting relevant information, mainly position and semantic, of the **A**reas **O**f **I**nterest present in the SVG. This project is initially part of a broader initiative to extract eye-tracking data from these same SVGs, the ultimate goal being to correlate gaze patterns with AOIs in order to better interpret the whole process one would use to understand various visualizations. With the aim of generalizing the approach, this project seeks to extract semantic information from SVG visualizations beyond the original — and already very verbose — dataset.

As discussed in the introduction, the original dataset was composed of very verbose SVGs. By that, we mean that these SVGs were greatly annotated, with many attributes, such as `ids` and `classes`, already having relevant semantic information. That is the first hypothesis made, parsing these attributes properly would already give a good idea of the semantics of the elements making up the visualizations. Combined with the tree structure of the SVG format itself [1], we initially thought that the original dataset could practically be fully interpreted this way. After discussing with our supervisor, we understood that the SVGs of the dataset had already been processed manually a lot for the sake of research, our *naive* approach was thus not satisfactory, and needed a more general strategy. We then made the hypothesis that the semantics of the AOIs could come from a *broader* context, which will be discussed in Section II.

Since our goal is also to extract positional information about the AOIs, we also needed a strategy to compute *bounding boxes*, in order to have a mapping of the semantic information in the space of the SVG. Even though the geometry is not trivial, the tree structure of the SVG combined with the vector description of the shapes made the computation of precise spatial boundaries a systematic, albeit complex, geometric problem. By traversing the document tree and translating the raw vector paths and affine transformations into exploitable geometric objects, we hypothesized that we could accurately compute these bounding boxes, regardless of the shape's complexity. This spatial mapping is a critical prerequisite: without a reliable system to map an element's extracted semantics to its exact 2D coordinates, correlating the data with eye-tracking logs would be impossible.

In the remainder of this report, we will detail our implementation pipeline, from the initial parsing of the SVG Document Object Model (DOM) to the geometric processing and generalized semantic extraction. We will then discuss the technical challenges encountered—particularly regarding non-standard SVG structures and complex composite shapes—before evaluating our findings and our alignment with the project's initial objectives.

II Implementation Process

In this section, we discuss both the technical and managerial aspect of our project's pipeline implementation.

II.1 Development Roles & Assigments

While we inittially thought that the best way to distribute the work was to assign to each of us a type of visualization, we quickly noticed that the divers visualizations had similar structure, and the work would be more efficiently distributed working on step of the global pipeline instead of everyone going throught the same processes over and over. This is a first major shift compared to what we had in the project plan. Thus, in definitive, we ended up with these assigments when it comes to the technical implementation:

- **Prototyping**: Charbel ABOU KHEIR, Jean Paul AL AM
- **Parsing**: Charbel ABOU KHEIR
- **Geometry & Bounding Boxes**: Jean Paul AL AM, Arsène MALLET
- **Semantics and AOI labels**: Marie-José TANNOURY, Alain TRAD
- **Pipelining**: Arsène MALLET
- **Visualization tools**: Marie-José TANNOURY

II.2 Technical Strategies

Let's now dive into the technical strategies and approaches made to tackle the AOIs extraction and analysis.

II.2.1 Pipeline overview

As discussed before, the system is organized as a linear pipeline, where each stage takes the output of the previous stage and adds more information to the SVG. The pipeline starts by parsing the SVG file into a node tree. Then, the system computes a bounding box for each node, assigns a semantic label, and links labeled data elements to their corresponding legend categories.

From the final labeled and categorized tree, the system produces two separate output representations. The first is a flat AOI table, and the second is a hierarchy JSON file, which reorganizes the same elements by chart section to make the output easier to inspect. Both outputs are generated from the same processed data, but neither one is derived from the other.

Each stage is implemented as a separate Python module. This made development and testing easier because each component could be checked independently. It also made the system more flexible during the project. For example, we were able to replace the semantic labeling module multiple times throughout the project timeline without changing the parsing or geometry code.

II.2.2 Parsing

The first stage reads the SVG file and builds a custom tree of node, more specifically SVGTreeNodeRaw objects, using Python's built-in `xml.etree` module. Each node stores its tag name, attribute dictionary, text content, parent reference, and ordered list of children. During construction, XML namespace prefixes are removed from tag names. This allows later modules to refer to SVG elements with simple names such as `rect` or `g`.

After the initial parse, the tree goes through a cleaning step. This step removes elements that do not carry visual meaning and will never become AOIs, including definitions, metadata, styles, scripts, clip paths, gradients, markers, and filters. It also removes the small set of tags not supported in the prototype, such as `tspan`, along with any groups that become empty after cleaning.

This cleaning step reduces the size of the tree before the more expensive stages begin. It also prevents the system from creating AOI entries for elements that are not rendered or not visible to the end user.

II.2.3 Geometric Bouding

Each node needs a bounding box in SVG coordinate space before semantic analysis can happen. The bounding box is used to localize chart elements and visualize them later. These bounding boxes are computed with the Shapely [2] library, using different logic depending on the SVG primitive type.

Rectangles and polygons are converted directly into Shapely shapes. Circles and ellipses are approximated as buffered points at a configurable resolution. Lines are given a small buffer so that zero-area

segments still produce a valid two-dimensional region. This is important for thin elements, such as axis ticks, which still need to be represented as AOIs.

Path elements require more processing than the other primitive types. The path data string in the d attribute is parsed using the `svg.path` library. Bezier curve segments are then approximated as straight-line segments sampled at a configurable resolution.

Container elements, such as `<g>` groups and the `<svg>` root, do not have their own explicit geometry. Instead, their bounding region is computed from their children. This is done bottom-up by taking the union of the children's polygons or descendant bounding boxes. If the union creates a disconnected shape, the system uses a convex hull instead.

SVG `transform` attributes, including `translate`, `rotate`, `scale`, and `matrix`, are parsed and applied to each element's bounding box in the order required by the SVG specification. This was important for elements such as rotated axis titles, where the displayed position is different from the element's original coordinate frame.

One limitation is that text bounding boxes are approximate. The geometry module uses the element's coordinate frame rather than the rendered glyph outlines, so text regions do not always match the visible characters exactly. One practical issue from this stage is discussed in Section III

II.2.4 Semantic Labeling

This stage assigns a semantic role to each node in the bounded tree. The labels come from a fixed set of sixteen roles: `data-mark`, `data-node`, `data-link`, `data-label`, `data-group`, `data-container`, `axis`, `axis-tick`, `axis-label`, `axis-title`, `grid`, `legend`, `legend-title`, `legend-item`, `legend-item-symbol`, `legend-item-label`, and `unknown`.

Semantic labeling changed the most during the project. We started with hardcoded rules for each chart type, then moved to confidence scoring, and finally replaced it with a deterministic priority chain. Each version fixed one issue but introduced or exposed another, which led to the final approach.

II.2.4.a Dataset Context

The dataset contains 16 hand-crafted SVG files grouped into four visual-encoding families:

- Group A includes geomaps, `A-a-1` and `A-a-2`, and treemaps, `A-b-1` and `A-b-2`, using color-coded regions;
- Group B includes lollipop charts, `B-a-1` and `B-a-2`, and scatter plots, `B-b-1` and `B-b-2`, relying on position and shape encoding;
- Group C includes choropleth maps, `C-a-1` and `C-a-2`, and calendar heatmaps, `C-b-1` and `C-b-2`, based on color-intensity encoding;
- Group D includes star maps, `D-a-1` and `D-a-2`, and knowledge graphs, `D-b-1` and `D-b-2`, using node-link encoding.

We also tested four Observable Plot SVGs: `Ba1Plot`, `Ba2Plot`, `Bb1Plot`, and `Bb2Plot`, covering regular and larger lollipop and scatter plots. These files came from a different charting library and used a different attribute style. The hand-crafted files often stored semantic information in descriptive `id` values or `data-name` attributes, while the Observable Plot files relied mainly on `aria-label`. This made them useful for testing whether the labeling method generalized beyond the original files. In total, the pipeline was developed and validated on 20 SVG files.

II.2.4.b Priority Chain

As discussed before, the following only presents the specifics of the final approach, the initial tests and strategies are nevertheless discussed further in Section III.1.

The final method uses a strict five-step priority chain. Each step is evaluated only if the previous steps do not assign a label, and the process stops once a label is found. This makes the output easier to debug because each label comes from one specific rule instead of a mixture of scores. Because semantic information appeared in different attributes depending on the SVG (e.g., `id` vs. `aria-label`), the final method scans attributes broadly, while ignoring attributes that mainly describe geometry, color, shape, or styling.

- Step 1: Handles clear cases. The root `<svg>` is labeled `data-container`, and `<foreignObject>` is labeled `legend`, since Observable Plot uses it to embed HTML-rendered legend content inside SVGs.
- Step 2: Scans the remaining attributes for keywords from a predefined keyword map. The map links substrings such as `axis`, `legend-title`, `cell-`, and `area-` to semantic labels. More specific keywords are checked before general ones, so `axis-title` is matched before `axis`.
- Step 3: Uses selective parent inheritance. A `<text>` element inside an axis group becomes an `axis-label`, a `<path>` or `<line>` inside an axis group becomes an `axis-tick`, and a `<rect>` inside a legend-item becomes a `legend-item-symbol`.
- Step 4: Uses sibling context. If a node has four or more siblings with the same shape tag and similar dimensions, it is likely to be a `data-mark`. This helps detect repeated visual elements such as calendar cells, scatter plot points, and map regions when semantic attributes are missing.
- Step 5: A conservative fallback. Text elements become `data-label`, shapes with saturated fill color become `data-mark`, and elements covering more than seventy percent of the canvas become `data-container`. Positional axis detection was explicitly excluded from this final version because it caused too many errors in earlier iterations.

II.2.4.c Category Linking

After semantic labeling, each `data-mark` has a type, but it does not yet have a specific identity. We also need to know which category that mark represents, such as which region on a map, which series in a scatter plot, or which food item in a lollipop chart.

This information is usually encoded in the chart legend, so the system runs a matching step immediately after labeling.

The matching step first extracts all `legend-item-symbol` elements and all `legend-item-label` elements from the full extracted data. It then pairs them by their order of appearance in the document. The first symbol is matched with the first label, the second symbol with the second label, and so on.

We considered sibling-based matching, but rejected it because it was not reliable across the SVGs in our dataset. In many files, intermediate group elements separate legend symbols from their labels in the DOM, so the symbol and label are not always direct siblings.

Each legend symbol is described with a shape signature. This signature includes the tag, the fill color normalized to an RGB tuple, the aspect ratio, and a path complexity measure for path elements. Color normalization converts CSS color formats into one common representation. This includes three-digit shorthand hex, six-digit hex, and named colors. As a result, values such as `#ed8` and `#eedd88` are recognized as equivalent.

Data marks are then matched against legend signatures using three fallback levels. The system first tries a full signature match. If that fails, it tries color alone, which is useful for map charts where the legend symbol might be a small rectangle but the data element is an irregular polygon. If that also fails, it uses tag and aspect ratio alone, which helps when marks share the same color but differ in shape.

Some charts in the dataset use two legends at the same time. For example, the `B-a-1` lollipop chart uses one legend to encode food item group by color and another legend to encode nutrient type by shape. In these cases, a single category string would lose information.

To handle this, the system groups legend symbols and labels under their nearest preceding `legend-title` element. It then builds a separate signature map for each legend group and matches each data mark against all groups independently. The resulting category field is a dictionary with one entry per legend group, such as `{"FOOD ITEM GROUP": "dishes", "TYPES OF NUTRIENTS": "Protein"}`. This preserves the full categorical information encoded in the chart.

II.3 Outputs

The pipeline produces three output files for each chart. The first is the flat AOI JSON, which lists all visible elements after filtering. Each element includes an ID, semantic label, bounding box, fill color, relevant text, category, and parent reference. For path elements, the original SVG path data is also saved so irregular shapes, such as choropleth regions, can be tested more precisely in future work.

The second output is the hierarchy JSON. It stores the same information, but groups it by chart section instead of keeping one flat list. Data marks and their labels are grouped under `data-container`, legend items are grouped under their legend title, and axis and grid elements each have their own section. Each element type only keeps the fields needed for it. For example, data marks keep their bounding box, fill color, and category, while text labels keep their text and bounding box. This makes the file easier to read when checking the chart structure. A short example can be found in Section Appendix A:.

The third output is a simplified SVG that shows the computed bounding boxes on top of the original chart. This was used as a visual check during development.

We also built a browser-based AOI viewer as a self-contained HTML file [3]. It loads the flat AOI JSON and the matching SVG, shows color-coded bounding boxes by label type, displays element details when clicked, and lets the user turn label types on or off.

III Challenges & Findings

III.1 Semantic Labeling Iterations & Generalization

Semantic labeling changed the most during the project. Before arriving at the deterministic priority chain detailed in Section II.2.4.b, we started with hardcoded rules for each chart type and then moved to confidence scoring. Each version fixed one issue but introduced or exposed another, which ultimately shaped our final approach.

III.1.1 First Iteration: Hardcoded Per-Chart-Type Rules

The earliest labeling approach did not try to generalize across chart types. Each of the eight chart types (Aa, Ab, Ba, Bb, Ca, Cb, Da, and Db) was manually inspected, and known `id` and `class` substrings were hardcoded for each type. For example, the geomap type (Aa) searched for `area-`, `region-`, and `label-container`, while the star map type (Da) searched for `lines-paths`, `line-`, and `all-*`.

The chart type was deduced from the filename. For example, `A-a-1.svg` was treated as type Aa, so only the Aa rules were applied. An element was considered meaningful if its id, class, or parent id matched one of the type-specific prefixes. Matched elements were then assigned to broad categories such as `legend`, `label`, `axis`, `data mark`, or `group` using a second generic keyword check.

The Challenge: This worked perfectly on the original hand-crafted files because the rules were written for those specific files. However, every new chart type required a new rule list, and the rules did not transfer well across chart types. This became painfully clear with the Observable Plot files, which did not follow the same `id` or `data-name` conventions and would have required an entirely separate rule set.

III.1.2 Second Iteration: Confidence Scoring

To reduce this dependence on chart-specific rules, the next version used confidence scoring. For each node, the system combined signals from keyword matches in aria-label, id, and class, the element's position on the canvas, groups of similarly sized repeating siblings, the parent's label, and fill-color saturation. Each signal added a floating-point score to possible labels. The scores were summed, and the label with the highest score above a threshold was selected. This seemed more flexible because the system could use whichever signals were present in each SVG and use visual clues to identify different labels rather than relying completely on IDs.

The Challenge: The main problem was that the scores were not always generalizable and signals started conflicting. Several weak signals could outweigh a clearer attribute-based signal. For example, background rectangles were often labeled as axis elements because they were near the bottom or left side of the canvas, even when they had no axis-related attributes. The scores that worked on one SVG file didn't work for the others. This made some labels unreliable because they came from accumulated weak evidence instead of a clear semantic clue. At the end of this step, we realized a critical finding: we must give the most importance to the attributes if they clearly state the label of the item, rather than adding additional scoring to them which may change the final outcome.

III.2 Visualization tools

In the project plan, we had anticipated that building visualization tools for the work we output would eventually be complicated and that we had to start early to be able to have enough time to end up with a satisfying tool. However, even with our initial concern, we prioritized the robustness of the core extraction pipeline, meaning the development of advanced visualization tools was left as a secondary objective. The web-based viewer [3] is a good initial approach and was useful to verify the work done, but it still lacks features for which time was lacking during implementation phase (such as manual label modification with automatic child-node reassignment, more advanced overlay capabilities for the original SVGs, or having more control of what's shown/hidden in the viewer.). In the end, we ended up with a viewer that we feel is a bit too minimal.

IV Results & Discussion

IV.1 Pipeline Validation & Performance

Overall, the final implementation successfully generalized the extraction of semantic Areas of Interest (AOIs) across both the 16 hand-crafted SVGs and the external Observable Plot visualizations. By abandoning rigid hardcoded rules and volatile confidence scoring in favor of the deterministic priority chain, we achieved a much higher reliability in semantic labeling.

The geometric pipeline also proved robust. By parsing affine transformations and traversing the DOM, the system correctly mapped complex vector shapes into precise bounding boxes using Shapely. This ensures that the spatial mapping prerequisite for the broader eye-tracking initiative is fully met. The outputs, both the hierarchical JSON and the flat AOI tables, are structured as needed for downstream cross-referencing.

IV.2 Alignment with the Proposed Plan

When looking back at our initial project plan, the core objective, building a set of automated tools to extract position and semantics from SVG visualizations, was successfully achieved. However, our methodology and team organization adapted significantly along the way.

Initially, we planned to distribute tasks by visualization type. As discussed in Section II, we quickly realized this was inefficient and pivoted to a pipeline-stage distribution. This proved to be a much more agile and effective strategy, allowing team members to specialize in parsing, geometry, or semantics independently.

Furthermore, our initial assumption that parsing native SVG attributes (`id` and `class`) would be sufficient was proven wrong. The necessity to iterate through three different semantic labeling strategies was not anticipated in the original timeline. While this caused delays in the labeling phase, it was a crucial technical pivot; without it, our pipeline would have severely overfitted to the provided hand-crafted dataset and failed entirely on external libraries like Observable Plot.

IV.3 Lessons Learned & Future Work

If we had to start the project over, we would prioritize the development of the browser-based AOI viewer much earlier in the cycle. Having a visual debugging tool from day one would have accelerated our realization that the confidence scoring model was mislabeling background elements, saving valuable development time. We also learned that in data extraction pipelines, definitive structural attributes must be prioritized over accumulated probabilistic signals.

Moving forward, the generated data structures provide a robust foundation for the eye-tracking correlation phase. Future work could focus on refining text bounding approximations—perhaps by integrating font-rendering engines to trace exact glyph outlines rather than relying on coordinate frames—and expanding the semantic keyword map to natively support a wider array of charting frameworks.

Appendices

Appendix A: Output Example

Below is a short example of a JSON file outputted by our pipeline:


```
{
   "data-container": {
  "data-mark-1": {
    "bbox": [639.07, 643.95, 745.03, 797.64],
    "fill": "#ed3584",
    "category": {"REGION": "IRONCLIFF"}
  }
   },
   "legend": {
  "REGION": {
```

```
    "legend-item-1": {"text": "IRONCLIFF", "fill": "#ed3584"}
  }
   }
}
```

Appendix B: AI Usage

In the spirit of transparency, and as demanded by the teaching staff, we decided to discuss our AI usage through everyone's own experience. Indeed, since everyone had different type of usage, we decided it was more meaningful to let everyone share their methods instead of having a *general/low-relevancy* overall description:

— Charbel ABOU KHEIR

— Jean Paul AL AM

Personally, my use of AI (Gemini 3.1, the Pro model most of the time) is mainly for idea development. By this, I mean that I share my initial thoughts on a process and ask the 'clanker' for its feedback, alternative approaches, and a general architectural view. I strictly avoid having it write raw code for me; it tends to overcomplicate things, use deprecated features, and I generally find it harder to prompt for exact implementation details than to just write it myself. Instead, when a process is clear in my head, or after I finish a first implementation, I feed it to the LLM to hunt for structural mistakes or edge cases. A concrete example from this project was the pipeline integration of the `geometry` module. My initial code did not quite fit the broader pipeline's API, so I asked the LLM for refactoring strategies, which helped me restructure the logic while still writing the code myself.

— Arsène MALLET

— Marie-José TANNOURY

— Alain TRAD

References

[1] World Wide Web Consortium (W3C), "Scalable Vector Graphics (SVG) Specification." [Online]. Available: https://www.w3.org/TR/SVG/

[2] S. Gillies and contributors, "Shapely: Manipulation and analysis of geometric objects." [Online]. Available: https://shapely.readthedocs.io/

[3] Charbel ABOU KHEIR, Jean Paul AL AM, Arsène MALLET, Marie-José TANNOURY, and Alain TRAD, "Simple AOI web-based AOI viewer." [Online]. Available: https://arsnm.github.io/svg_aoi_analysis/